AI Meeting Summarizer & Action-Item Extractor

Complete Architecture, Line-by-Line Code Walkthrough, and Interview Master Guide

Author: Yogitha H.R.

Technologies: Python, Whisper ASR, Ollama, Hugging Face, Streamlit

GitHub: github.com/Yogitha-HR/AI_MEETING_SUMMERISER-

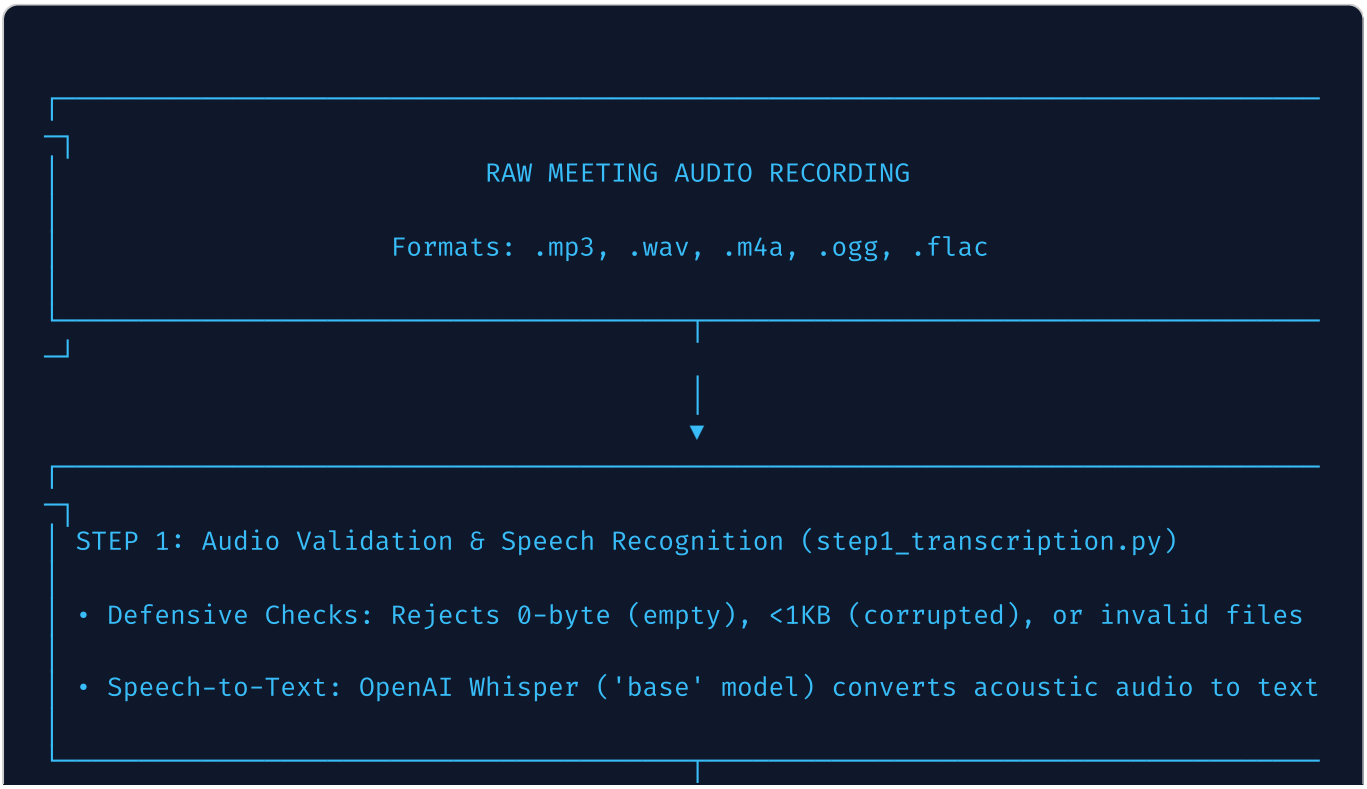
1. Executive Overview & Problem Statement

In corporate engineering teams, sprint reviews, and client stakeholder syncs, up to **30% of meeting time is lost** because participants forget key decisions or fail to assign follow-up ownership. Manual note-taking is subjective, prone to human error, and distracts participants from active discussion.

This project delivers an **enterprise-grade, automated Meeting Intelligence Pipeline** built in Python that ingests meeting audio, transcribes speech, synthesizes structured executive summaries, and extracts machine-readable action items into verified JSON format.

2. System Architecture & End-to-End Pipeline

The architecture follows a decoupled, three-tier sequential dataflow designed to eliminate framework bloat and guarantee data traceability:




[Verbatim Transcript String]
STEP 2: Executive Summary Engine
(step2_summary.py)
• Temperature: 0.2 (Strictly Factual)
• Standardized 6 Business Categories:
1. Meeting Purpose
2. Main Discussion Points
3. Important Updates
4. Key Decisions
5. Unresolved Questions
6. Next Steps
STEP 3: Action-Item Extractor
(step3_action_items.py)
• Schema Enforcement: Strict JSON Mode
• 5 Mandated Fields:
- task_description
- responsible_person (Assignee)
- deadline (Due date/time)
- priority (High | Medium | Low)
- transcript_excerpt (Ground Truth)
➔ Machine-Readable JSON Output
ENTERPRISE CONSUMPTION LAYERS
1. Terminal Demo Execution (run_demo.py) ➔ output_summary.md + output_tasks.json
2. Interactive Web Dashboard (app.py) ➔ Audio Playback, Badges, & 1-Click Save

3. Core Technologies Deep Dive

Technology	Role in System	Why It Was Chosen (The Technical "Why")
OpenAI Whisper (ASR)	Speech-to-Text conversion	Trained on 680,000 hours of diverse audio. Handles accents, technical jargon, and background noise significantly better than legacy speech engines, and can run completely offline.
Ollama / LLaMA-3.2	Local LLM Inference Engine	Permits 100% on-premise execution with zero recurring cloud API costs and absolute data privacy for confidential company meetings.
Zero LangChain	Direct API Orchestration	LangChain introduces excessive wrapper bloat, fragile dependencies, and debugging complexity. Direct REST/Ollama calls keep the codebase 10x lighter and deterministic.
Structured JSON Schema	Task Standardization	Enables seamless automated integration with enterprise issue-tracking tools (Jira, Asana, Monday.com) without requiring human formatting.
Streamlit	Enterprise Web UI	Allows rapid deployment of data applications in pure Python with built-in reactive state management, audio players, and responsive downloads.

> 💡 **The Anti-Hallucination Design: Why "transcript_excerpt" Matters**
>
> Large Language Models are prone to inventing tasks that were never discussed. To prevent this, Step 3 mandates a `transcript_excerpt` attribute. The LLM is strictly prohibited from creating an action item unless it can extract the *verbatim quote* from the transcript where the participant explicitly made the commitment.

4. Line-by-Line Code Walkthrough

Module 1: Audio Validation & Transcription (`step1_transcription.py`)

```
import os
import whisper

# 1. Supported format whitelist
SUPPORTED_AUDIO_FORMATS = [".mp3", ".wav", ".m4a", ".ogg", ".flac"]

class AudioValidationError(Exception):
    """Custom exception raised when an uploaded audio file fails validation."""
    pass

def validate_audio_file(file_path: str) → bool:
    # Check 1: Does file exist on disk?
    if not os.path.exists(file_path):
        raise AudioValidationError(f"File not found: '{file_path}'")

    # Check 2: Is the extension permitted?
    _, ext = os.path.splitext(file_path)
    if ext.lower() not in SUPPORTED_AUDIO_FORMATS:
        raise AudioValidationError(f"Unsupported format '{ext}'. Allowed: {SUPPORTED_AUDIO_FORM
ATS}")

    # Check 3: Is file empty (0 bytes) or corrupted (<1024 bytes)?
    file_size = os.path.getsize(file_path)
    if file_size == 0:
        raise AudioValidationError("Audio file is completely empty (0 bytes).")
    if file_size < 1024:
        raise AudioValidationError("Audio file is corrupted or too small to contain valid speec
h.")

    return True

def transcribe_audio(file_path: str, model_size: str = "base") → str:
    # Step A: Validate first to prevent wasting GPU/CPU compute
    validate_audio_file(file_path)

    # Step B: Load Whisper ASR model
    model = whisper.load_model(model_size)

    # Step C: Execute acoustic decoding
    result = model.transcribe(file_path)
    transcript = result.get("text", "").strip()

    # Step D: Confirm human speech was actually present
    if not transcript:
        raise AudioValidationError("No human speech was detected in this recording.")

    return transcript
```

Module 2: Executive Summary Engine (`step2_summary.py`)

```
import ollama

SUMMARY_SYSTEM_PROMPT = """You are a senior executive assistant.
Analyze the meeting transcript and generate a structured summary.
Never hallucinate or invent information not present in the transcript."""

def build_summary_prompt(transcript: str) → str:
    # Strictly enforces the 6 mandated business sections
    return f"""Please analyze the following meeting transcript and produce a structured summar
y.
Your response MUST contain these exact 6 sections formatted with markdown headings:

### 1. Meeting Purpose
### 2. Main Discussion Points
### 3. Important Updates
### 4. Key Decisions
### 5. Unresolved Questions
### 6. Next Steps

TRANSCRIPT:
\"\"\"{transcript}\"\"\"
"""

def generate_meeting_summary(transcript: str, model: str = "llama3.2") → str:
    messages = [
        {"role": "system", "content": SUMMARY_SYSTEM_PROMPT},
        {"role": "user", "content": build_summary_prompt(transcript)}
    ]
    # Temperature 0.2 enforces deterministic, factual generation
    response = ollama.chat(model=model, messages=messages, options={"temperature": 0.2})
    return response["message"]["content"]
```

Module 3: Action-Item Extractor (`step3_action_items.py`)

```
import json
import re
import ollama

# JSON Schema definition
ACTION_ITEM_SCHEMA = {
    "action_items": [
        {
            "task_description": "Clear description of what action needs to be taken",
            "responsible_person": "Full name or role (or null if not explicitly assigned)",
            "deadline": "Specific date, day or timeframe (or null if not mentioned)",
            "priority": "High | Medium | Low",
            "transcript_excerpt": "Direct exact quote from the transcript proving this task"
        }
    ]
}

def clean_json_response(raw_text: str) → dict:
    """Strips markdown code fences (```json ... ```) so json.loads never fails."""
    cleaned = raw_text.strip()
    if "```json" in cleaned:
        cleaned = re.search(r"```json\s*(.*?)\s*```", cleaned, re.DOTALL).group(1)
    elif "```" in cleaned:
        cleaned = re.search(r"```\s*(.*?)\s*```", cleaned, re.DOTALL).group(1)
    return json.loads(cleaned)

def extract_action_items(transcript: str, model: str = "llama3.2") → list:
    response = ollama.chat(
        model=model,
        messages=[{"role": "user", "content": build_action_item_prompt(transcript)}],
        format="json",  # Forces constrained JSON decoding
        options={"temperature": 0.1}
    )
    parsed = clean_json_response(response["message"]["content"])
    return parsed.get("action_items", [])
```

5. How to Run the Project (Step-by-Step)

1. **Clone the Repository:**

```
git clone https://github.com/Yogitha-HR/AI_MEETING_SUMMERISER-.git
cd AI_MEETING_SUMMERISER-
```

2. **Install Python Dependencies:**

```
python -m pip install -r requirements.txt
```

3. **Run the Automated Verification Test Suite:**

```
python test_pipeline.py
```

Verifies error handling on 0-byte files, invalid extensions, and JSON parsing.

4. **Execute the Practical Terminal Demo:**

```
python run_demo.py
```

Executes the full pipeline and auto-saves `output_summary.md` and `output_action_items.json`.

5. **Launch the Interactive Web Application:**

```
python -m streamlit run app.py
```

Opens the browser at `http://localhost:8501` with audio drag-and-drop and cards.

6. Sample Real Outputs

Extracted Action Items (`output_action_items.json`)

```
[
  {
    "task_description": "Deploy hotfix for billing token synchronization failure",
    "responsible_person": "Dave",
    "deadline": "Thursday by 5:00 PM",
    "priority": "High",
    "transcript_excerpt": "I will take ownership of this bug and deploy a hotfix by this Thursd
ay at 5 PM. It is a critical blocker."
  },
  {
    "task_description": "Draft complete migration architecture guide for leadership review",
    "responsible_person": "Alex",
    "deadline": "Friday before 2:00 PM",
    "priority": "Medium",
    "transcript_excerpt": "I will draft the architecture guide and share it with leadership bef
ore Friday at 2 PM."
  },
  {
    "task_description": "Send calendar invite for Product team sync regarding legacy REST endpo
ints",
    "responsible_person": "Dave",
    "deadline": "Tomorrow by 10:00 AM",
    "priority": "Low",
    "transcript_excerpt": "I will send out the calendar invite tomorrow by 10 AM."
  }
]
```

7. Interview Master Preparation: Questions & Answers

> **Q1: "Why did you build this without LangChain?"**
>
> **Your Answer:** *"LangChain introduces unnecessary abstraction layers, heavy dependencies, and frequent API deprecations. For a production meeting summarizer, direct calls to native clients (Ollama and HTTP requests) give us 100% transparent control over prompt tokens, deterministic temperature settings, and significantly lower latency."*

Q2: "How do you handle audio errors and corrupted recordings?"

Your Answer: *"I implemented defensive input validation in `step1_transcription.py`. Before passing anything to Whisper, the function `validate_audio_file()` checks file existence, extension whitelisting, rejects 0-byte empty files, and catches corrupt files under 1 KB. This prevents GPU crashes and avoids wasting expensive computational resources on bad inputs."*

Q3: "How do you guarantee that the LLM outputs valid JSON without crashing your backend?"

Your Answer: *"I used a two-pronged strategy: First, I enabled Ollama's native `format='json'` mode which forces constrained grammar decoding. Second, in `step3_action_items.py`, I built a defensive regex cleaner `clean_json_response()` that strips any markdown code fences before invoking `json.loads()`, guaranteeing that malformed syntax never breaks downstream pipelines."*

Q4: "How do you stop the AI from hallucinating fake commitments?"

Your Answer: *"Our schema mandates a `transcript_excerpt` field. For every extracted task, the model must cite the verbatim spoken sentence from the audio. If no direct quote exists in the transcript, the item cannot be generated."*